

AI-based Flood Prediction & Early Warning System

Domain: Disaster Management | AI/ML Based Proof of Concept (POC)

1. Problem Statement

2022 ki floods ne Pakistan mein bohat bara jani aur mali nuksan kiya. Timely aur localized warning systems ki kami hai, jiski wajah se evacuation aur relief efforts mein delay hota hai.

2. Proposed AI/ML Solution

- Rainfall data, river water level, aur historical flood records collect kiye jayenge.
- ML model (Random Forest) flood risk score predict karega specific regions ke liye.
- FastAPI backend high-risk areas ke liye alert-worthy data serve karega.
- Haversine formula se user ki location aur nearest flood-risk zone ke beech distance calculate hoga.

3. Tools & Technologies

Scikit-learn, Pandas, FastAPI (backend), PostgreSQL (region/alert data), Folium (map visualization), Haversine formula

4. Dataset

Pakistan Meteorological Department (PMD) data, WAPDA river flow data, open global flood datasets

5. Feasibility for POC

Medium — publicly available proxy datasets se achi working POC demo banayi ja sakti hai.

6. Expected Impact

Advance warning se evacuation time milega, jani-mali nuksan kam hoga, disaster response better resource allocation kar sakegi.

Technical Defense — Sir Q&A Preparation

7. Kya Use Karoon Gi (My Tech Stack) & Kyun

- Random Forest — tabular rainfall/river-level data ke liye robust aur interpretable hai.
- Haversine Formula — user location aur nearest at-risk zone ke beech real distance (lat/long) nikalne ke liye.
- FastAPI + PostgreSQL — flood-risk zones aur alerts ka structured data store/serve karne ke liye.
- Logging — real-time alerts trigger hone par sab kuch log karoon gi audit trail ke liye.

8. Kya Use NAHI Karoon Gi & Kyun (Rejected Alternatives)

- Complex LSTM as first choice — POC ke liye Random Forest se start (data limited hai), LSTM future scope.
- Real-time IoT sensors — abhi scope se bahar, publicly available PMD/WAPDA data se hi demonstrate karoon gi.
- Paid SMS gateway real integration — sirf demo/mock alert system POC mein dikhaon gi.

9. Kaise Implement Karoon Gi (Step-by-Step)

- Step 1: Historical rainfall aur river-level data collect/clean karna.
- Step 2: Random Forest classifier train karna region-wise flood-risk predict karne ke liye.
- Step 3: Haversine formula implement karna nearest risk-zone distance nikalne ke liye.
- Step 4: FastAPI endpoint jo location input lekar nearest risk return kare.
- Step 5: Map-based demo (Folium) jahan risk zones aur user location dikhein.

10. Possible Issues / Challenges & Mitigation

- Localized real-time river data Pakistan mein publicly kam available hai.
- False alarms ka risk — conservative thresholds set karoon gi.
- Model ko regularly retrain karna hoga naye data ke sath.

11. Anticipated Questions from Sir & Answers

Q: Haversine formula kyun use ki?

A: Do geographic points ke beech real-world distance (great-circle distance) calculate karne ke liye standard approach hai.

Q: Random Forest kyun, LSTM kyun nahi is baar?

A: Available historical data limited hai; Random Forest chhote/tabular data par zyada reliable hai.

Q: Alerts kaise deliver hongy real system mein?

A: POC mein demo/mock alert dikhaon gi; real deployment mein Twilio SMS API integrate ki ja sakti hai.

Project Architecture & Folder Structure

12. Folder Structure (Sir ke pattern par based)

```
flood_prediction_system/
├── src/
│   ├── flood_zones/
│   │   ├── __init__.py
│   │   ├── models.py          # FloodZone table (id, region, lat, lng, risk_level, rainfall, river_level)
│   │   ├── schemas.py
│   │   ├── router.py          # GET /flood-zones/nearest, GET /flood-zones/risk
│   │   ├── services.py        # Haversine distance + risk prediction logic
│   │   ├── utils.py           # haversine_distance() function
│   │   └── alerts/
│   │       ├── __init__.py
│   │       ├── models.py      # Alert table (id, zone_id, message, sent_at)
│   │       ├── schemas.py
│   │       ├── router.py
│   │       └── services.py
│   ├── database.py
│   ├── settings.py
│   ├── main.py
│   └── ml_model/
│       └── flood_risk_model.pkl
├── logs/
│   └── app.log
├── streamlit_app.py
├── requirements.txt
├── .env
├── .gitignore
├── notes.ipynb
└── README.md
```

13. Har File Mein Kya Likha Hoga

- **models.py** — SQLAlchemy table definition (e.g. flood_zones table: id, columns, timestamps). Yeh DB structure define karta hai.
- **schemas.py** — Pydantic models jo request body aur response ka shape/validation define karte hain (e.g. Create schema, Response schema).
- **router.py** — FastAPI endpoints (@router.get, @router.post, etc.) yahan define hote hain — yeh sirf request receive karke services ko call karta hai.
- **services.py** — Business logic yahan hoti hai: DB query build karna, execute karna, result process karna, phir router ko return karna.
- **utils.py** — Helper/reusable functions (e.g. image preprocessing, prediction formatting, common validation) jo ek se zyada jagah use hote hain.
- **database.py** — SQLAlchemy engine, SessionLocal, aur Base declaration — DB connection setup yahan hota hai.
- **settings.py** — .env se DATABASE_URL, API keys, model paths load karta hai (Pydantic Settings/BaseSettings).
- **main.py** — FastAPI app instance banata hai, sab routers include karta hai (app.include_router), aur entry point hai (uvicorn isi ko run karta hai).
- **utils.py (haversine_distance function)** — do lat/long points lekar kilometers mein distance return karta hai, jo nearest-zone lookup mein use hota hai.

14. Data Flow — Sab Kaise Connect Hota Hai

- User Streamlit app se location/rainfall input deta hai (image/data) → request FastAPI ke router.py tak jati hai.
- router.py request ko services.py ko forward karta hai (router sirf routing karta hai, logic nahi rakhta).
- services.py business logic run karta hai: model se prediction lena, ya database query karna (SQLAlchemy session use karke).
- Agar DB involved hai, services.py database.py ke SessionLocal se connect hota hai aur models.py ki table mein data insert/fetch karta hai.
- Result schemas.py ke response model ke mutabiq format hoke wapis router se Streamlit/client ko JSON response mein jata hai.
- Har step par logging aur try-except exception handling hoti hai taake error properly track ho aur 200/500 status code sahi return ho.

15. pgAdmin Mein Kya Data Show Hoga

- pgAdmin mein connect karne ke baad, database ke andar **flood_zones** table dikhega jab main.py chalega aur SQLAlchemy models create_all() run karega.
- Table ke columns: id, region_name, latitude, longitude, risk_level, rainfall_mm, river_level, updated_at.
- Query Tool se 'SELECT * FROM flood_zones;' likh ke run karne par saara stored data (predictions/records) rows ke form mein dikhega.
- Jaise jaise API calls (POST requests) hongi, naye rows real-time mein is table mein add hote jayenge — yeh live demo mein sir ko dikhaya ja sakta hai.

16. Streamlit Interface Mein Kya Dikhega

- Streamlit app browser mein khulega (streamlit run streamlit_app.py) — ek simple web page jismein title aur upload/input section hoga.
- User apni location (lat/long ya city) upload/enter karega → 'Predict/Submit' button dabayega.
- Streamlit background mein FastAPI ke endpoint ko requests.post() se call karega (jaisa maine API integration tasks mein practice ki).
- Result screen par display hoga: flood risk ka prediction/output, confidence score, aur agar history feature hai to past records ki list bhi.

17. uvicorn Chalane Par Kaunsi APIs Show Hongi (/docs)

- uvicorn src.main:app --reload chalane ke baad, browser mein http://127.0.0.1:8000/docs kholne par Swagger UI dikhega.
- Wahan '/flood-zones' prefix ke andar sab endpoints list honge: GET /flood-zones/nearest?lat=..&lng;=..., GET /flood-zones/risk, POST /alerts, GET /health.
- Har endpoint 'Try it out' se directly test kiya ja sakta hai — request body dal ke 'Execute' karne par response (200 success ya error) turant dikhega.
- /redoc par same APIs ka clean documentation view bhi milta hai.

18. Additional Anticipated Questions from Sir

Q: Haversine formula ka code kahan rakhoge?

A: flood_zones/utils.py mein ek reusable haversine_distance(lat1, lon1, lat2, lon2) function banaoon gi, jise services.py import karke use karega — jaisa 'utils mein one-time ya multiple-use functions' wala pattern sir ne sikhaya.

Q: Alerts module alag kyun rakha?

A: Single Responsibility Principle follow karne ke liye — flood_zones module risk-prediction handle karta hai, alerts module sirf notification-sending logic handle karta hai, dono independent aur maintainable rehte hain.

Q: Agar PMD API available na ho to demo kaise chalega?

A: Historical CSV dataset ko fallback ke tor par PostgreSQL mein pre-load kar rakhoon gi, taake live API na hone par bhi demo static-but-realistic data se chal sake.